

Разделение ролей

Я бы предложил такую схему на 4 участников, где у каждого есть свой главный deliverable и минимум блокировок от других.

Участник	Основная зона	Что делает	Главный результат
1	Data pipeline	Скачивает ACDC, изучает структуру, пишет preprocessing для image-level labels из panoptic annotations, делает train/val metadata	dataset_builder.py, labels.csv/json, описание классов
2	Model pipeline	Поднимает dual-head модель на MambaVision, dataloader, losses, training loop, чекпоинты	model.py, train.py, конфиги обучения
3	Evaluation & baselines	Делает метрики, threshold tuning для multi-label head, baseline с ResNet-50, сравнение MambaVision-T/S	eval.py, таблицы результатов, графики
4	Repo & report/presentation	Собирает структуру репозитория, README, воспроизводимость, слайды/отчёт, сводит результаты команды	README.md, report/slides, финальная упаковка

Такое деление естественно соответствует вашим deliverables: preprocessing script, trained dual-head model, report with metrics, final presentation.^{[2] [1]}

Как работать параллельно

Чтобы не ждать друг друга, разбейте проект на два слоя: общий контракт данных и независимые модули поверх него. MambaVision уже имеет понятный classification entry point и pretrained variants вроде T и S, поэтому моделист может начать сразу на временном dummy metadata, пока дата-инженер дописывает финальную разметку.^[1]

Нужен единый контракт:

- Формат metadata: image_path, weather_label, multi_hot_12_classes, split.
- Фиксированный список 12 объектов и кодировка классов.
- Единые названия экспериментов и папок: configs/, src/, reports/, artifacts/.

Тогда параллельно можно делать так:

1. Участник 1 быстро выпускает v0 таблицу примеров и схему данных.
2. Участник 2 сразу пишет dataset class, heads, losses и train loop на v0.
3. Участник 3 сразу готовит evaluation code и baseline template.
4. Участник 4 сразу делает каркас repo, README, шаблон слайдов и таблицу для занесения результатов.

Детализация по людям

Участник 1 — Data/labels

Его зона самая критичная в начале, но потом почти автономная. ACDC публикуется как benchmark adverse conditions, а идея вывести image-level presence labels из существующих panoptic annotations выглядит обоснованной именно потому, что исходный датасет содержит pixel-level panoptic annotations.^[2] ^[1]

Задачи:

- Скачать и разложить ACDC по структуре.
- Проверить split train/val/test.
- Написать mapping panoptic classes → 12 image-level object labels.
- Сделать sanity checks: сколько изображений на класс, class imbalance, co-occurrence.
- Передать команде metadata_train.csv, metadata_val.csv, class_map.json.

Участник 2 — Архитектура и обучение

Этому человеку лучше отдать core model. В официальной репозитории есть pretrained MambaVision-T и MambaVision-S, а также простой путь загрузки модели через mambavision или Hugging Face, что снижает риски для двухнедельного проекта.^[3] ^[1]

Задачи:

- Shared backbone: MambaVision-T как основная модель.
- Две головы: 4-class softmax head и 12-class sigmoid head.
- Loss: $\text{CE}(\text{weather}) + \lambda * \text{BCE}(\text{objects})$.
- Частичное замораживание backbone на старте.
- Логирование train/val losses и checkpointing.
- Подготовить 1 стабильный training recipe, который все потом используют.

Участник 3 — Метрики и эксперименты

Это отдельный поток, и его реально вести независимо после появления первых checkpoint'ов. Ваше описание метрик логично: для weather head — accuracy/F1/confusion matrix; для object head — mAP, per-class F1, Hamming loss и tuning thresholds.^[2]

Задачи:

- Написать evaluation pipeline.
- Подобрать thresholds для 12 multi-label классов.
- Сделать baseline ResNet-50.
- Сделать ablation: MambaVision-T vs MambaVision-S vs ResNet-50.

- Посчитать performance отдельно по fog/night/rain/snow, потому что ACDC специально организован по adverse conditions.^[2]

Участник 4 — Репозиторий и презентация

Это не “оформитель”, а интегратор. Он снижает хаос и собирает итоговую историю проекта: задача, данные, метод, трудности, результаты. По вашему заданию нужен репозиторий с кодом и report/presentation, где быстро понятны problem, approach, difficulties, results и графики losses/quality.^[2]

Задачи:

- Создать repo structure и правила именования.
- Поддерживать README: setup, dataset prep, train, eval.
- Собирать figures: loss curves, metric charts, confusion matrix.
- Подготовить 8–12 слайдов или короткий report.
- Финально свести вклад каждого и проверить воспроизводимость.

План по срокам

У вас дедлайн темы был к 14 мая, а сдача — к 28 мая, так что сейчас нужен очень жёсткий двухнедельный спринт.

Предлагаю такой план:

- 16–18 мая: data contract, repo skeleton, preprocessing v1, training skeleton.
- 19–22 мая: первый обученный MambaVision-T, первая оценка, первые графики.
- 23–25 мая: baseline ResNet-50, MambaVision-S, threshold tuning, per-weather breakdown.
- 26–27 мая: финальная таблица результатов, README, слайды.
- 28 мая: финальная проверка и submit.

Что зафиксировать сегодня

Сегодня вам лучше сразу утвердить 6 вещей, иначе работа “в параллель” быстро превратится в конфликт версий:

- Кто owner каждого блока.
- Кто maintainer главной ветки репозитория.
- Формат metadata и список 12 классов.
- Главная метрика проекта: например, weather accuracy + object mAP.
- Minimal viable result: одна обученная модель MambaVision-T и один baseline ResNet-50.
- Stretch goal: MambaVision-S, asymmetric loss, per-weather ablation.

Практически я бы назвал роли так:

- Person A — Data Engineer
- Person B — Modeling Engineer
- Person C — Evaluation Researcher
- Person D — Integration & Presentation Lead

Ниже — короткая формулировка, которую можно прямо отправить в чат команды:

- Участник 1: preprocessing ACDC и генерация image-level labels.
- Участник 2: dual-head MambaVision model и обучение.
- Участник 3: evaluation, baselines, ablations, threshold tuning.
- Участник 4: repo integration, README, report/presentation, final submission.

Если хотите, я могу следующим сообщением сделать ещё **готовый task board в формате GitHub Projects/Trello** с задачами по каждому участнику на 2 недели.



1. <https://acdc.vision.ee.ethz.ch>
2. <https://arxiv.org/abs/2104.13395>
3. <https://github.com/nvlab/mambavision>
4. <https://www.semanticscholar.org/paper/ACDC:-The-Adverse-Conditions-Dataset-with-for-Scene-Sakaridis-Dai/787b9f2a81412a05cb26269a1e2740c746192f04>
5. <https://github.com/MIC-DKFZ/ACDC2017>
6. https://openaccess.thecvf.com/content/CVPR2025/papers/Hatamizadeh_MambaVision_A_Hybrid_Mamba-Transformer_Vision_Backbone_CVPR_2025_paper.pdf
7. <https://github.com/lsmcolab/Semantic-Segmentation-under-Adverse-Conditions>
8. <https://acdc.vision.ee.ethz.ch/download>
9. <https://github.com/caojiaxi0505>
10. <https://www.creatis.insa-lyon.fr/Challenge/acdc/databases.html>
11. <https://www.creatis.insa-lyon.fr/Challenge/acdc/>
12. <https://www.kaggle.com/datasets/anhoangvo/acdc-dataset>
13. https://openaccess.thecvf.com/content/ICCV2021/papers/Sakaridis_ACDC_The_Adverse_Conditions_Dataset_With_Correspondences_for_Semantic_Driving_ICCV_2021_paper.pdf
14. <https://huggingface.co/datasets/msepulvedagodoy/acdc>
15. <https://huggingface.co/nvidia/MambaVision-L-21K>